SCIF30005 Forest Fire Mini Project

Student ID: *2225197*
Student Name: *Zhaoyang Yu*

February 26, 2026

Abstract

This report presents a parallel implementation of a forest fire cellular automaton in C++ with MPI and analyses both its statistical behaviour and its parallel performance. The model is simulated on square grids of size $N \times N$ with initial tree occupation probability p, where trees in the top row are ignited and fire spreads using a von Neumann neighbourhood. Three main questions are studied: (i) how the probability of the fire reaching the bottom depends on p, (ii) how many repeat runs M are needed for convergence of averaged quantities, and (iii) how the MPI implementation scales with the number of tasks on BluePebble.

The results show a sharp transition in the probability of reaching the bottom around $p \approx 0.59$, which becomes steeper for the larger grid size. The average burn duration peaks near the transition region, indicating longer-lived fires when large connected clusters form. Convergence analysis shows that $M = 50$ is often sufficient for qualitative conclusions, but larger M is needed near the transition for tighter confidence intervals. Strong-scaling results show substantially better speedup for $N = 500$ than for $N = 100$, with performance limited at high task counts by halo communication and per-step collective synchronisation.

1 Introduction

The forest fire model is a simple cellular automaton that exhibits threshold-like behaviour and is often used as an accessible example of how local rules can produce complex global dynamics. In this project, a square grid is populated randomly with trees, a fire is ignited at the top, and the simulation evolves in discrete time until no cells are burning. The key observable of interest is whether the fire reaches the bottom row, which behaves similarly to a percolation-style spanning event.

This project has two scientific aims and one computational aim. First, the dependence of the model outcomes on the tree density p is studied, including the probability that the fire reaches the bottom and the average number of timesteps before the fire extinguishes. Second, convergence with respect to the number of repeat runs M is investigated, with particular attention to values of p near the transition region where fluctuations are largest. Third, the model is parallelised using MPI and tested on BluePebble to analyse strong-scaling performance for two grid sizes ($N = 100$ and $N = 500$).

2 Methods

2.1 Forest fire model

The simulation uses a square grid of size $N \times N$. Each cell is either empty, a living tree, burning, or dead. The initial condition is generated by independently placing a tree in each cell with probability p. At time $t = 0$, all living trees in the top row are set on fire.

The model then evolves in discrete timesteps according to the following rules:

- A living tree ignites if at least one of its von Neumann neighbours (up, down, left, right) is burning.
- A burning tree becomes dead in the next timestep.
- Empty cells remain empty and dead cells remain dead.

The simulation terminates when no cells are burning. For each run, two outputs are recorded:

1. the number of timesteps until extinction;
2. whether the fire reached the bottom row at any point.

2.2 MPI parallel implementation

The model is implemented in C++ and parallelised using MPI. A one-dimensional domain decomposition is used, where the grid is split into contiguous blocks of rows across MPI tasks. Each task stores its local rows together with ghost rows to represent neighbouring data required for updates near subdomain boundaries.

At each timestep, neighbouring tasks exchange boundary rows using blocking `MPI_Sendrecv` calls. Two collective reductions are also performed per timestep:

- a global reduction to determine whether any task still has burning cells;
- a global reduction to determine whether the fire has reached the bottom row.

This design is straightforward and correct, but introduces synchronisation overhead that becomes more significant at high task counts and small problem sizes.

2.3 Statistical estimates and experimental design

For each parameter set (N, p), the simulation is repeated for M independent runs using different random seeds. The estimated probability of reaching the bottom is computed as

$$\hat{P}(\text{reach bottom}) = \frac{1}{M} \sum_{m=1}^{M} I_m,$$

where I_m is a Bernoulli indicator for the mth run. The average number of steps is also recorded. Error bars in the plots represent 95% confidence intervals estimated from repeated independent batches generated in the analysis pipeline.

Three groups of experiments were performed:

1. parameter sweeps over $p \in [0.4, 0.8]$ for $N = 100$ and $N = 500$;
2. convergence tests near the transition region ($p = 0.58, 0.60, 0.62$);
3. strong-scaling tests at fixed $p = 0.6$ using multiple MPI task counts.

For the science plots, each figure is presented separately rather than as a paired subfigure comparison. This is because the computational settings differ between the $N = 100$ and $N = 500$ experiments due to runtime constraints. Within each individual figure, however, only the stated horizontal-axis variable is changed, while the remaining simulation and computational settings are held fixed.

3 Results

3.1 Transition in the probability of reaching the bottom

For each probability sweep, only p is varied within the graph; N, M, the number of batches used to estimate uncertainty, the MPI task count, and the model rules are all fixed within that figure.

Figure 1 shows the estimated probability that the fire reaches the bottom row as a function of p for $N = 100$. A clear transition is observed: for small p, the fire almost never spans the system, while for sufficiently large p, reaching the bottom becomes almost certain. The main change occurs near $p \approx 0.59$. For example, the estimated spanning probability rises from approximately 0.23 at $p = 0.58$ to 0.68 at $p = 0.60$ and 0.95 at $p = 0.62$.

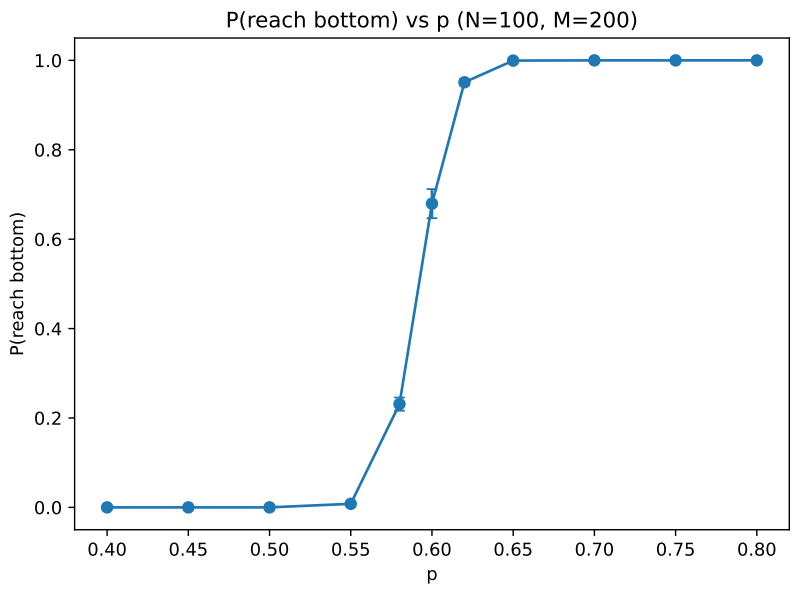

Figure 1: Estimated probability that the fire reaches the bottom row versus tree density p for $N = 100$. **Varied:** p. **Controlled within this figure:** $N = 100$, $M = 200$ repeats per batch, $n_{\text{runs}} = 10$ independent batches, 8 MPI tasks, top-row ignition, von Neumann neighbourhood, and termination when no cells are burning. Error bars show 95% confidence intervals across batches.

Figure 2 shows the same observable for $N = 500$. The overall behaviour is similar, but the transition is visibly sharper. This is consistent with finite-size effects: larger systems smooth the transition less, so the change from subcritical to supercritical behaviour becomes more abrupt. In this case the probability is still close to zero at $p = 0.58$, rises to around 0.93 at $p = 0.60$, and reaches 1 by $p = 0.62$.

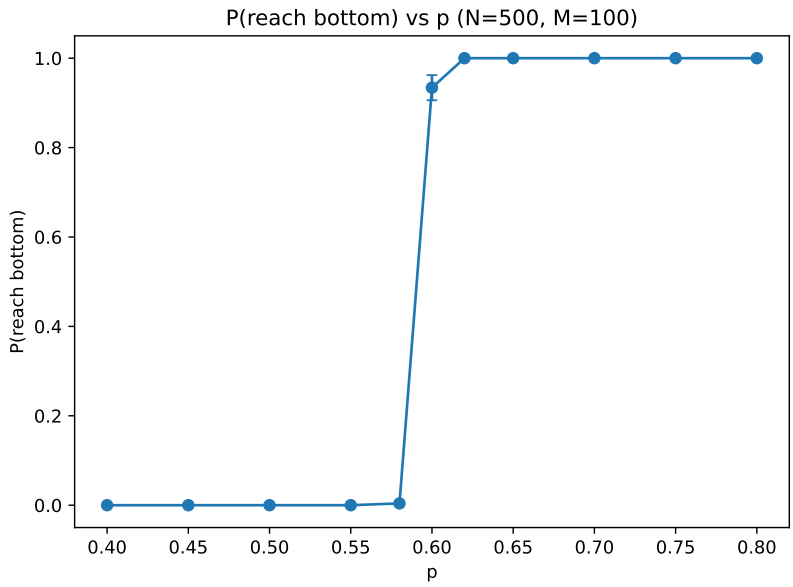

Figure 2: Estimated probability that the fire reaches the bottom row versus tree density p for $N = 500$. **Varied:** p. **Controlled within this figure:** $N = 500$, $M = 100$ repeats per batch, $n_{\text{runs}} = 5$ independent batches, 16 MPI tasks, top-row ignition, von Neumann neighbourhood, and termination when no cells are burning. Error bars show 95% confidence intervals across batches.

Although the two figures are not intended as a strict like-for-like computational comparison, the common threshold region near $p \approx 0.59$ is evident in both, and the larger grid shows a sharper transition.

3.2 Average burn duration and peak near the transition

Within each of the following analyses, only p is varied, while the grid size, repeat count, batch structure, MPI task count, and model rules are fixed.

Figure 3 shows the average number of timesteps until extinction for $N = 500$. The mean burn duration increases sharply as p approaches the transition region, reaches a peak near $p = 0.60$, and then decreases again. This pattern occurs because, near the threshold, the fire propagates through large irregular connected clusters that keep the system active for much longer. The same qualitative behaviour is also seen for $N = 100$, but with a much smaller peak.

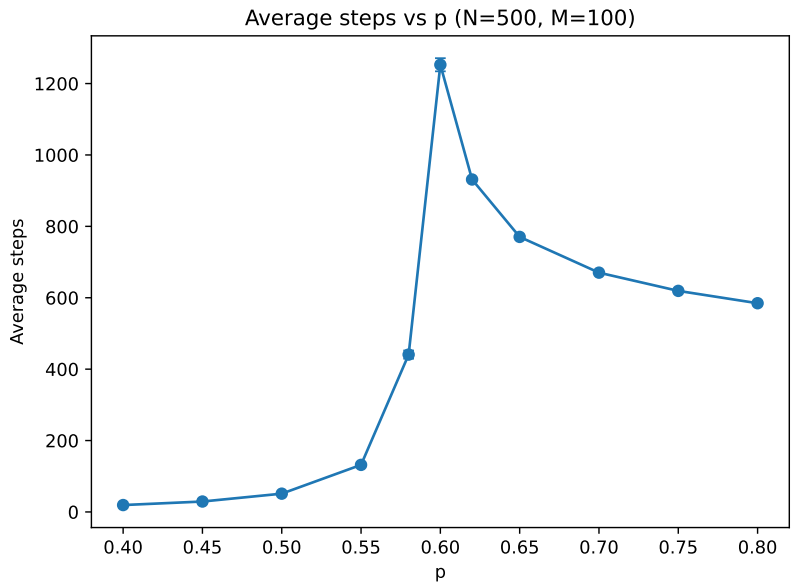

Figure 3: Mean burn duration (timesteps until extinction) versus tree density p for $N = 500$. **Varied:** p. **Controlled within this figure:** $N = 500$, $M = 100$ repeats per batch, $n_{\text{runs}} = 5$ independent batches, 16 MPI tasks, and fixed model rules. Error bars show 95% confidence intervals across batches.

For $N = 100$, the burn duration also increases near the transition and peaks around $p = 0.62$, but the maximum is much smaller than for $N = 500$. This supports the interpretation that larger grids allow the fire to explore longer and more complicated connected structures before extinction.

3.3 Convergence with respect to the number of repeats M

For the convergence study, only M is varied within the graph. The grid size, the selected probability values, the MPI task count, and the model rules are fixed within the figure.

Figure 4 shows the convergence of $\hat{P}(\text{reach bottom})$ for $N = 100$ at $p = 0.58$, 0.60, and 0.62. The estimated means are fairly stable once M reaches about 50–100, but the confidence intervals continue to shrink as M increases. The uncertainty is clearly largest at $p = 0.60$, which is expected because this is closest to the transition and therefore has the greatest run-to-run variability. Therefore, $M = 50$ may be sufficient for a broad qualitative conclusion, but larger M is preferable near the threshold if a tighter estimate is required.

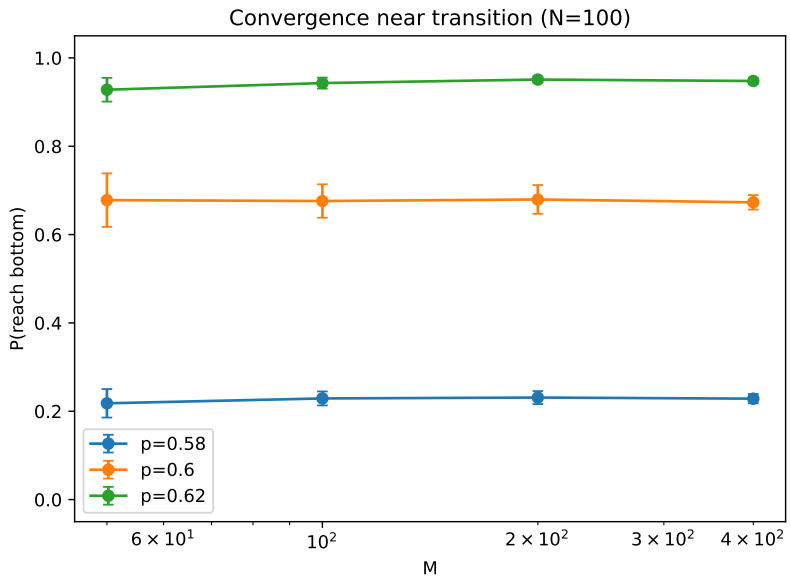

Figure 4: Convergence of $\hat{P}$(reach bottom) with increasing repeat count M for $N = 100$. **Varied:** M. **Controlled within this figure:** $N = 100$, $p \in \{0.58, 0.60, 0.62\}$, $n_{\text{runs}} = 10$ independent batches, 8 MPI tasks, and fixed model rules. Error bars show 95% confidence intervals across batches.

3.4 Strong-scaling performance

In Figure 5, the only quantity varied is the number of MPI tasks. For each curve, N and the simulation parameters are held fixed, and speedup is measured relative to the corresponding 1-task runtime.

The larger grid ($N = 500$) achieves substantially better scaling than the smaller grid ($N = 100$), which is expected because the compute-to-communication ratio is higher for larger subdomains. For $N = 100$, speedup improves initially but saturates and eventually degrades at high task counts. This indicates that communication and synchronisation overheads dominate once the local workload becomes too small. In contrast, for $N = 500$, speedup continues to increase up to 16 tasks, although it remains sublinear. The gap from ideal linear scaling can be attributed to halo exchanges at each timestep and the cost of two global `MPI_Allreduce` operations per timestep.

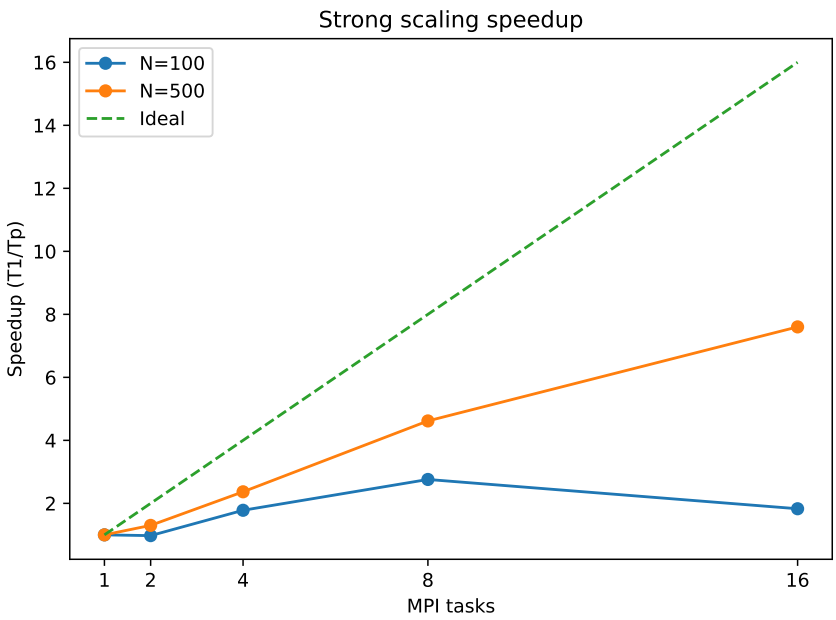

Figure 5: Strong-scaling speedup versus MPI task count at fixed $p = 0.60$. **Varied:** number of MPI tasks. **Controlled within this figure:** one curve for $N = 100$ and one curve for $N = 500$, with fixed $p = 0.60$, fixed averaging procedure for each curve, and the same MPI implementation and timing definition within each curve. The dashed line shows ideal linear speedup.

4 Discussion and limitations

The results are consistent with the expected qualitative behaviour of a forest fire / percolation-like model. The probability of reaching the bottom exhibits a threshold near $p \approx 0.59$, and the average number of steps peaks in the same region. The sharpening of the transition with increasing grid size supports the interpretation that finite-size effects smooth the transition for smaller systems.

The convergence analysis highlights an important practical point: the number of repeats M required depends on what quantity is being estimated and how much uncertainty is acceptable. A single rule such as "$M = 50$ is enough" is too simplistic. Instead, convergence should be judged relative to the uncertainty in the estimated observable, particularly near the transition where run-to-run variation is largest.

The performance results also show a clear trade-off between parallelism and overhead. The current implementation is robust and conceptually simple, but it performs halo exchange every timestep and requires global collective communication for termination and bottom-reach detection. These collective operations can become a bottleneck, especially for small problem sizes or high task counts.

A further limitation is that only a one-dimensional row-wise decomposition was used. While this is appropriate for the current grid sizes and project scope, more advanced decomposition strategies may improve scaling for larger problems.

5 Future work

Possible improvements include using a formal convergence criterion based on CI width, reducing synchronisation overhead in the MPI implementation, exploring non-blocking communication, and extending the study to additional grid sizes or alternative neighbourhood definitions.

6 Conclusion

A parallel C++/MPI forest fire model was implemented and used to study both statistical behaviour and computational performance. The simulations show a clear transition in the probability of the fire reaching the bottom near $p \approx 0.59$, with a sharper transition for larger grid size. The average burn duration peaks near the transition, indicating longer-lived dynamics in the near-critical regime.

Convergence with respect to the number of repeat runs depends on p: $M = 50$ may be sufficient for qualitative conclusions, but larger M is needed near the transition to reduce uncertainty. Strong-scaling tests show that the implementation provides useful speedup for larger grids, while smaller grids saturate earlier due to communication and synchronisation overheads.

Use of AI Tools

AI tools (including ChatGPT) were used for grammar checking, code debugging, bluepebble helping.

Code and Supplementary Materials

The assessed C++ source code submitted for this project contains the main simulation implementation. For reproducibility and completeness, a supplementary GitHub repository is also provided, containing the complete C++/MPI codebase, Python plotting scripts used to generate figures, Slurm/bash job scripts used for BluePebble runs, and supplementary workflow files.

Repository link: `https://github.com/GoJeonPa021/Year3_Mini_Project`

References

- SCIF30005 Forest Fire Mini Project specification and workshop materials.
- MPI documentation (for `MPI_Sendrecv` and `MPI_Allreduce`).